

Analytical problem solving

K. Yoganand
190525173

```
#include <stdio.h>

struct node {
    int pid, day, sales;
    struct node * next;
};

struct node * head = NULL;

void insert (int pid, int day, int sales)
{
    struct node * temp;
    temp = (struct
node *) malloc (Size of ( struct node));

    temp -> pid = pid;
    temp -> day = day;
    temp -> sales = sales;
    temp -> next = head;
    head = temp;
}

void highest sales() {
    struct node * p = head;
    int max = 0, pid, day;

    while (p != NULL) {
        if (p -> sales > max) {
            max = p -> sales;
            pid = p -> pid;
            day = p -> day;
        }
    }
}
```

```

} p = p->next;
} printf("\n Highest sale: \n");
printf("product ID = %d \n Day = %d \n sales = %d \n", prod, day, max);
}
int main() {
    insert(101, 1, 20);
    insert(101, 2, 35);
    insert(102, 1, 40);
    insert(102, 2, 25);
    insert(103, 3, 50);
    display();
    highestsale();
    return 0;
}

```

output:-

Sales enter:

product : 103 Day : 3 sales : 50

product : 102 Day : 2 sales : 25

product : 102 Day : 1 sales : 40

product : 101 Day : 2 sales : 35

product : 101 Day : 1 sales : 20

Highest sale:

product ID = 103

Day = 3

sales = 50

```
#include <stdio.h>
```

```
int main() {
```

```
    char postfix[] = "82+64-*2/";
```

```
    int stack[20], top = -1;
```

```
    int i, a, b;
```

```
    for (i = 0; postfix[i] != '\0'; i++)
```

```
{
```

```
    char ch = postfix[i];
```

```
    if (ch >= '0' && ch <= '9') {
```

```
        stack[++top] = ch - '0';
```

```
    }
```

```
    else {
```

```
        b = stack[top--];
```

```
        a = stack[top--];
```

```
        switch (ch) {
```

```
            case '+': stack[++top] =
```

```
                a + b; break;
```

```
            case '-': stack[++top] =
```

```
                a - b; break;
```

```
            case '*': stack[++top] =
```

```
                a * b; break;
```

```
            case '/': stackstack[++top] =
```

```
                a / b; break;
```

```
        }
```

```
    }
```

```
    printf("Infix expression :  
(8+2) *(64/21n");
```

```
printf("postfix expression :  
8 2 + 6 4 - * 2 / 1 4");  
printf("Result = %d", stack[top]);  
return 0;
```

}

output:-

infix expression : $(8+2) * (6+4) / 2$

postfix expression : $8\ 2\ +\ 6\ 4\ -\ *\ 2\ /\$

Result = 10

